Notebook 2 - Exploring and understanding data - Answers

This lesson is adapted from the Data Analysis and visualisation in R for Ecologists 3 lesson (https://datacarpentry.github.io/R-ecology-lesson/how-r-thinks-about-data.html)

Questions

- How does R store and represent data?

Objectives

- Explore the structure and content of data.frames
- Understand vector types and missing data
- Use vectors as function arguments
- Create and convert factors
- Understand how R assigns values to objects

This is an R Markdown (http://rmarkdown.rstudio.com) Notebook. When you execute code within the notebook, the results appear beneath the code.

You can execute code chunks by clicking the *Run* button within the chunk or by placing your cursor inside it and pressing `Ctrl + Shift + Enter` or OS X: `Cmd + Shift + Enter`. To execute a specific line within a code chunk, place your cursor on that line and press `Ctrl + Enter` or OS X: `Cmd + Enter`.

```
# try running this code chunk

3 + 2
```

```
## [1] 5
```

```
3 * 2
```

```
## [1] 6
```

You can add a new chunk of code in your language of choice by clicking the *Insert Chunk* button on the toolbar or by pressing `Ctrl + Alt + I` or OS X: `Cmd + Option + I`.

Tip - View source or visual mode

The R Markdown notebook (e.g.*'exploring_data_answers.Rmd'*) can be viewed or edited in either in *'Source'* mode which shows the raw Markdown commands, or in the *'Visual'* mode which shows how the Markdown will look in its formatted form. You can switch between the *'Source'* and *'Visual'* modes at any time using the tabs at the top of the notebook.

Tip - Preview

When you save the notebook, an HTML file containing the code and *output* will be saved alongside it. To preview the HTML file in the 'Viewer' tab click the *Preview* button or press `Ctrl + Shift + K` or OS X: `Cmd + Shift + K`. The preview shows you a rendered HTML copy of the current contents of the editor. Consequently, *Preview* does not run any R code chunks. Instead, the output of the chunk from when it was last run in the editor is displayed. There are other more sophisticated ways to display your notebook, such as using the *Knit* command.

1. Setup

We start by loading the required packages that you installed during the setup, **`tidyverse`** and **`ratdat`** packages.

If you do not have the packages installed, you can run `install.packages("tidyverse")` and `install.packages("tidyverse")` in the console.

It is a good practice not to put `install.packages()` into a script. This is because every time you run that whole script, the package will be reinstalled, which is typically unnecessary. You want to install the package to your computer once, and then load it with `library()` in each script where you need to use it.

```
library(tidyverse)
```

```
## ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
## ✔ dplyr     1.1.4     ✔ readr     2.1.5
## ✔ forcats   1.0.0     ✔ stringr   1.5.1
## ✔ ggplot2   3.5.2     ✔ tibble    3.3.0
## ✔ lubridate 1.9.4     ✔ tidyr     1.3.1
## ✔ purrr     1.0.4
## ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()    masks stats::lag()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become e
rrors
```

```
library(ratdat)
```

2. Exploring data

2.1 The data.frame

We spent quite a bit of time learning how to create visualisations from the `complete_old` data in the notebook *'visualisation_ggplot.Rmd'*, but we did not talk much about **what** this `complete_old` thing is. It's important to understand how R thinks about, represents, and stores data in order for us to have a productive working relationship with R.

The `complete_old` data is stored in R as a **data.frame**, which is the most common way that R represents tabular data (data that can be stored in a table format, like a spreadsheet).

A data frame is the representation of data in the format of a table where the columns are vectors that all have the same length. Because columns are vectors, each column must contain a single type of data (e.g., characters, integers, factors). For example, here is a figure depicting a data frame comprising a numeric, a character, and a logical vector.

We'll talk more about vectors shortly. For now, we can check what `complete_old` is by using the `class()` function:

```
class(complete_old)
```

```
## [1] "tbl_df"     "tbl"        "data.frame"
```

We can view the first few rows with the `head()` function, and the last few rows with the `tail()` function:

```
# view first and last rows of data

head(complete_old)
```

record_id <int>	mo... <int>	d... <int>	year <int>	plot_id <int>	species_id <chr>	s... <chr>	hindfoot_length <int>	weight <int>	genus <chr>
1	7	16	1977	2	NL	M	32	NA	Neotoma
2	7	16	1977	3	NL	M	33	NA	Neotoma
3	7	16	1977	2	DM	F	37	NA	Dipodomys
4	7	16	1977	7	DM	M	36	NA	Dipodomys

record_id <int>	mo… <int>	d… <int>	year <int>	plot_id <int>	species_id <chr>	s… <chr>	hindfoot_length <int>	weight <int>	genus <chr>
5	7	16	1977	3	DM	M	35	NA	Dipodomys
6	7	16	1977	1	PF	M	14	NA	Perognathus

6 rows | 1-10 of 13 columns

```
tail(complete_old)
```

record_id <int>	mo… <int>	d… <int>	year <int>	plot_id <int>	species_id <chr>	s… <chr>	hindfoot_length <int>	weight <int>	genus <chr>
16873	12	5	1989	8	DO	M	37	51	Dipodomys
16874	12	5	1989	16	RM	F	18	15	Reithrodontomys
16875	12	5	1989	5	RM	M	17	9	Reithrodontomys
16876	12	5	1989	4	DM	M	37	31	Dipodomys
16877	12	5	1989	11	DM	M	37	50	Dipodomys
16878	12	5	1989	8	DM	F	37	42	Dipodomys

6 rows | 1-10 of 13 columns

We used these functions with just one argument, the object `complete_old`, and we didn't give the argument a name, like we often did with `ggplot2`. In R, a function's arguments come in a particular order, and if you put them in the correct order, you don't need to name them. In this case, the name of the argument is `x`, so we can name it if we want, but since we know it's the first argument, we don't need to.

To learn more about a function, you can type a `?` in front of the name of the function, which will bring up the official documentation for that function:

```
# function help

?head
```

Tip - Help

In the *Help* tab you can see the help that you just called

Tip - Documentation and information

Function documentation is written by the authors of the functions, so they can vary pretty widely in their style and readability. The first section of function documentation is **Description**, which gives you a concise description of what the function does, but it may not always be enough. The **Arguments** section defines all the arguments for the function and is usually worth reading thoroughly. Finally, the **Examples** section at the end will often have some helpful examples that you can run to get a sense of what the function is doing.

Another great source of information is **package vignettes**. Many packages have vignettes, which are like tutorials that introduce the package, specific functions, or general methods. You can run `vignette(package = "package_name")` to see a list of vignettes in that package. Once you have a name, you can run `vignette("vignette_name", "package_name")` to view that vignette. You can also use a web browser to go to `https://cran.r-project.org/web/packages/package_name/vignettes/` where you will find a list of links to each vignette (replace 'package_name' with the name of the package you are interested in). Some packages will have their own websites, which often have nicely formatted vignettes and tutorials.

Finally, learning to search for help is probably the most useful skill for any R user. The key skill is figuring out what you should actually search for. It's often a good idea to start your search with `R` or `R programming`. If you have the name of a package you want to use, start with `R package_name`.

Many of the answers you find on the internet will be from a website called *Stack Overflow*, where people ask programming questions and others provide answers. It is generally poor form to ask duplicate questions, so before you decide to post your own, do some thorough searching to see if it has been answered before (it likely has). If you do decide to post a question on *Stack Overflow*, or any other help forum, you will want to create a **reproducible example** or **reprex**. If you are asking a complicated question requiring your own data and a whole bunch of code, people probably won't be able or willing to help you. However, if you can hone in on the specific thing you want help with, and create a minimal example using smaller, fake data, it will be much easier for others to help you. If you search for `'how to make a reproducible example in R'`, you will find some great resources to help you out.

When using R functions, some arguments are optional. For example, the `n` argument in `head()` specifies the number of rows to print. It defaults to 6, but we can override that by specifying a different number:

```
# increase number of rows to view

head(complete_old, n = 10)
```

record_id <int>	mo... <int>	d... <int>	year <int>	plot_id <int>	species_id <chr>	s... <chr>	hindfoot_length <int>	weight <int>	genus <chr>
1	7	16	1977	2	NL	M	32	NA	Neotoma
2	7	16	1977	3	NL	M	33	NA	Neotoma
3	7	16	1977	2	DM	F	37	NA	Dipodomys
4	7	16	1977	7	DM	M	36	NA	Dipodomys
5	7	16	1977	3	DM	M	35	NA	Dipodomys
6	7	16	1977	1	PF	M	14	NA	Perognathus
7	7	16	1977	2	PE	F	NA	NA	Peromyscus
8	7	16	1977	1	DM	M	37	NA	Dipodomys
9	7	16	1977	1	DM	F	34	NA	Dipodomys
10	7	16	1977	6	PF	F	20	NA	Perognathus

1-10 of 10 rows | 1-10 of 13 columns

If we order them correctly, we don't have to name the arguments either:

```
# increase number of rows to view, without naming arguments

head(complete_old, 10)
```

record_id <int>	mo... <int>	d... <int>	year <int>	plot_id <int>	species_id <chr>	s... <chr>	hindfoot_length <int>	weight <int>	genus <chr>
1	7	16	1977	2	NL	M	32	NA	Neotoma
2	7	16	1977	3	NL	M	33	NA	Neotoma
3	7	16	1977	2	DM	F	37	NA	Dipodomys
4	7	16	1977	7	DM	M	36	NA	Dipodomys
5	7	16	1977	3	DM	M	35	NA	Dipodomys
6	7	16	1977	1	PF	M	14	NA	Perognathus
7	7	16	1977	2	PE	F	NA	NA	Peromyscus
8	7	16	1977	1	DM	M	37	NA	Dipodomys
9	7	16	1977	1	DM	F	34	NA	Dipodomys
10	7	16	1977	6	PF	F	20	NA	Perognathus

1-10 of 10 rows | 1-10 of 13 columns

Additionally, if we name them, we can put them in any order we want:

```
# increase number of rows to view, naming arguments

head(n = 10, x = complete_old)
```

record_id <int>	mo… <int>	d… <int>	year <int>	plot_id <int>	species_id <chr>	s… <chr>	hindfoot_length <int>	weight <int>	genus <chr>
1	7	16	1977	2	NL	M	32	NA	Neotoma
2	7	16	1977	3	NL	M	33	NA	Neotoma
3	7	16	1977	2	DM	F	37	NA	Dipodomys
4	7	16	1977	7	DM	M	36	NA	Dipodomys
5	7	16	1977	3	DM	M	35	NA	Dipodomys
6	7	16	1977	1	PF	M	14	NA	Perognathus
7	7	16	1977	2	PE	F	NA	NA	Peromyscus
8	7	16	1977	1	DM	M	37	NA	Dipodomys
9	7	16	1977	1	DM	F	34	NA	Dipodomys
10	7	16	1977	6	PF	F	20	NA	Perognathus

1-10 of 10 rows | 1-10 of 13 columns

Generally, it's good practice to start with the required arguments, like the data.frame whose rows you want to see, and then to name the optional arguments. If you are ever unsure, it never hurts to explicitly name an argument.

Let's get back to investigating our `complete_old` data.frame. We can get some useful summaries of each variable using the `summary()` function:

```
# summary of data

summary(complete_old)
```

```
##    record_id         month             day            year         plot_id
##  Min.   :    1   Min.   : 1.000   Min.   : 1.0   Min.   :1977   Min.   : 1.00
##  1st Qu.: 4220   1st Qu.: 3.000   1st Qu.: 9.0   1st Qu.:1981   1st Qu.: 5.00
##  Median : 8440   Median : 6.000   Median :15.0   Median :1983   Median :11.00
##  Mean   : 8440   Mean   : 6.382   Mean   :15.6   Mean   :1984   Mean   :11.47
##  3rd Qu.:12659   3rd Qu.: 9.000   3rd Qu.:23.0   3rd Qu.:1987   3rd Qu.:17.00
##  Max.   :16878   Max.   :12.000   Max.   :31.0   Max.   :1989   Max.   :24.00
##
##   species_id            sex            hindfoot_length     weight
##  Length:16878       Length:16878       Min.   : 6.00   Min.   :  4.00
##  Class :character   Class :character   1st Qu.:21.00   1st Qu.: 24.00
##  Mode  :character   Mode  :character   Median :35.00   Median : 42.00
##                                        Mean   :31.98   Mean   : 53.22
##                                        3rd Qu.:37.00   3rd Qu.: 53.00
##                                        Max.   :70.00   Max.   :278.00
##                                        NA's   :2733    NA's   :1692
##     genus             species              taxa            plot_type
##  Length:16878       Length:16878       Length:16878       Length:16878
##  Class :character   Class :character   Class :character   Class :character
##  Mode  :character   Mode  :character   Mode  :character   Mode  :character
##
##
##
##
```

And, as we have already done, we can use `str()` to look at the structure of an object:

```
# structure of data

str(complete_old)
```

```
## tibble [16,878 × 13] (S3: tbl_df/tbl/data.frame)
##  $ record_id      : int [1:16878] 1 2 3 4 5 6 7 8 9 10 ...
##  $ month          : int [1:16878] 7 7 7 7 7 7 7 7 7 7 ...
##  $ day            : int [1:16878] 16 16 16 16 16 16 16 16 16 16 ...
##  $ year           : int [1:16878] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 ...
##  $ plot_id        : int [1:16878] 2 3 2 7 3 1 2 1 1 6 ...
##  $ species_id     : chr [1:16878] "NL" "NL" "DM" "DM" ...
##  $ sex            : chr [1:16878] "M" "M" "F" "M" ...
##  $ hindfoot_length: int [1:16878] 32 33 37 36 35 14 NA 37 34 20 ...
##  $ weight         : int [1:16878] NA NA NA NA NA NA NA NA NA NA ...
##  $ genus          : chr [1:16878] "Neotoma" "Neotoma" "Dipodomys" "Dipodomys" ...
##  $ species        : chr [1:16878] "albigula" "albigula" "merriami" "merriami" ...
##  $ taxa           : chr [1:16878] "Rodent" "Rodent" "Rodent" "Rodent" ...
##  $ plot_type      : chr [1:16878] "Control" "Long-term Krat Exclosure" "Control" "Rodent Exclosu
re" ...
```

We get quite a bit of useful information here. First, we are told that we have a data.frame of 16878 observations, or rows, and 13 variables, or columns.

Next, we get a bit of information on each variable, including its type (`int` or `chr`) and a quick peek at the first 10 values. You might ask why there is a `$` in front of each variable. This is because the `$` is an operator that allows us to select individual columns from a data.frame.

The `$` operator also allows you to use what's called 'tab-completion' to quickly select which variable you want from a given data.frame. For example, to get the `year` variable, we can type `complete_old$` and then hit `Tab` . We get a list of the variables that we can move through with up and down arrow keys. Hit `Enter` when you reach `year` , which should finish this code:

```
# subset variable

complete_old$year
```

```
##     [1] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##    [15] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##    [29] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##    [43] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##    [57] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##    [71] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##    [85] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##    [99] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [113] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [127] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [141] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [155] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [169] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [183] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [197] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [211] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [225] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [239] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [253] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [267] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [281] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [295] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [309] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [323] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [337] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [351] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [365] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [379] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [393] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [407] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [421] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [435] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [449] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [463] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [477] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977
##   [491] 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1977 1978
##   [505] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [519] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [533] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [547] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [561] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [575] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [589] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [603] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [617] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [631] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [645] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [659] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [673] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [687] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [701] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [715] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [729] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [743] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [757] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [771] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [785] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [799] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [813] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [827] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [841] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [855] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [869] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
##   [883] 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978 1978
```

```
## [16129] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16143] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16157] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16171] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16185] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16199] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16213] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16227] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16241] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16255] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16269] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16283] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16297] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16311] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16325] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16339] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16353] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16367] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16381] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16395] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16409] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16423] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16437] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16451] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16465] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16479] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16493] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16507] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16521] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16535] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16549] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16563] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16577] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16591] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16605] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16619] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16633] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16647] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16661] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16675] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16689] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16703] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16717] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16731] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16745] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16759] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16773] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16787] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16801] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16815] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16829] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16843] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16857] 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989 1989
## [16871] 1989 1989 1989 1989 1989 1989 1989 1989
```

What we get back is a whole bunch of numbers, the entries in the `year` column printed out in order.

2.2 Vectors: the building block of data

You might have noticed that our last result looked different from when we printed out the `complete_old` data.frame itself. That's because it is not a data.frame, it is a **vector**. A vector is a 1-dimensional series of values, in this case a vector of numbers representing years.

Data.frames are made up of vectors; each column in a data.frame is a vector. Vectors are the basic building blocks of all data in R. Basically, everything in R is a vector, a bunch of vectors stitched together in some way, or a function. Understanding how vectors work is crucial to understanding how R treats data, so we will spend some time learning about them.

There are 4 main types of vectors (also known as *atomic vectors*):

1. `"character"` for strings of characters, like our `genus` or `sex` columns. Each entry in a character vector is wrapped in quotes. In other programming languages, this type of data may be referred to as "strings".
2. `"integer"` for integers. All the numeric values in `complete_old` are integers. You may sometimes see integers represented like `2L` or `20L`. The `L` indicates to R that it is an integer, instead of the next data type, `"numeric"`.
3. `"numeric"`, aka `"double"`, vectors can contain numbers that are decimals. Other languages may refer to these as "float" or "floating point" numbers.
4. `"logical"` for `TRUE` and `FALSE`, which can also be represented as `T` and `F`. In other contexts, these may be referred to as "Boolean" data.

Vectors can only be of a **single type**. Since each column in a data.frame is a vector, this means an accidental character following a number, like `29,` can change the type of the whole vector. Mixing up vector types is one of the most common mistakes in R, and it can be tricky to figure out. It's often very useful to check the types of vectors.

To create a vector from scratch, we can use the `c()` function, and put values inside, separated by commas.

```
# create a numeric vector

c(1, 2, 5, 12, 4)
```

```
## [1]  1  2  5 12  4
```

As you can see, those values get printed out in the console, just like with `complete_old$year`. To store this vector so we can continue to work with it, we need to assign it to an object.

```
# assign vector to an object

num <- c(1, 2, 5, 12, 4)
```

Tip - Assignment operator

In RStudio, typing `Alt + -` will write `<-` in a single keystroke or OS X: `Option + -`

For historical reasons, you can also use `=` for assignments, but not in every context. *Note* `<-` assigns values to your global environment, i.e. they can be used throughout your scripts.

You can check what kind of object `num` is with the `class()` function.

```
# check the class of the object

class(num)
```

```
## [1] "numeric"
```

We see that `num` is a `numeric` vector.

Tip - Environment tab

In the *Environment* tab you can also see the type for all vectors in memory.

Let's try making a character vector:

```
# create a character vector

char <- c("apple", "pear", "grape")
class(char)
```

```
## [1] "character"
```

Remember that each entry, like `"apple"`, needs to be surrounded by quotes, and entries are separated with commas. If you do something like `"apple, pear, grape"`, you will have only a single entry containing that whole string.

Finally, let's make a logical vector:

```
# create a logical vector

logi <- c(TRUE, FALSE, TRUE, TRUE)
class(logi)
```

```
## [1] "logical"
```

Challenge 1.a: Coercion

Since vectors can only hold one type of data, something has to be done when we try to combine different types of data into one vector.

What type will each of these vectors be? Try to guess without running any code at first, then run the code and use `class()` to verify your answers.

```
num_logi <- c(1, 4, 6, TRUE)
num_char <- c(1, 3, "10", 6)
char_logi <- c("a", "b", TRUE)
tricky <- c("a", "b", "1", FALSE)
```

```
class(num_logi)
```

```
## [1] "numeric"
```

```
class(num_char)
```

```
## [1] "character"
```

```
class(char_logi)
```

```
## [1] "character"
```

```
class(tricky)
```

```
## [1] "character"
```

Challenge 1.b: Coercion

R will automatically convert values in a vector so that they are all the same type, a process called **coercion**.

How many values in `combined_logical` are `"TRUE"` (as a character)?

```
combined_logical <- c(num_logi, char_logi)
```

```
combined_logical
```

```
## [1] "1"    "4"    "6"    "1"    "a"    "b"    "TRUE"
```

```
class(combined_logical)
```

```
## [1] "character"
```

Only one value is `"TRUE"`. Coercion happens when each vector is created, so the `TRUE` in `num_logi` becomes a `1`, while the `TRUE` in `char_logi` becomes `"TRUE"`. When these two vectors are combined, R doesn't remember that the `1` in `num_logi` used to be a `TRUE`, it will just coerce the `1` to `"1"`.

Challenge 1.c: Coercion

Now that you've seen a few examples of coercion, you might have started to see that there are some rules about how types get converted. There is a hierarchy to coercion. Can you draw a diagram that represents the hierarchy of what types get converted to other types?

`logical → integer → numeric → character`

Logical vectors can only take on two values: `TRUE` or `FALSE`. Integer vectors can only contain integers, so `TRUE` and `FALSE` can be coerced to `1` and `0`. Numeric vectors can contain numbers with decimals, so integers can be coerced from, say, `6` to `6.0` (though R will still display a numeric `6` as `6`.). Finally, any string of characters can be represented as a character vector, so any of the other types can be coerced to a character vector.

Coercion is not something you will often do intentionally; rather, when combining vectors or reading data into R, a stray character that you missed may change an entire numeric vector into a character vector. It is a good idea to check the `class()` of your results frequently, particularly if you are running into confusing error messages.

2.3 Missing data

One of the great things about R is how it handles missing data, which can be tricky in other programming languages. R represents missing data as `NA`, without quotes, in vectors of any type. Let's make a numeric vector with an `NA` value:

```
# create numeric vector with NA

weights <- c(25, 34, 12, NA, 42)
```

R doesn't make assumptions about how you want to handle missing data, so if we pass this vector to a numeric function like `min()`, it won't know what to do, so it returns `NA`:

```
# minimum function

min(weights)
```

```
## [1] NA
```

This is a very good thing, since we won't accidentally forget to consider our missing data. If we decide to exclude our missing values, many basic math functions have an argument to **rem**ove them:

```
# minimum function, remove NAs

min(weights, na.rm = TRUE)
```

```
## [1] 12
```

2.4 Vectors as arguments

A common reason to create a vector from scratch is to use in a function argument. The `quantile()` function will calculate a quantile for a given vector of numeric values. We set the quantile using the `probs` argument. We also need to set `na.rm = TRUE`, since there are `NA` values in the `weight` column.

```
# calculate quantile

quantile(complete_old$weight, probs = 0.25, na.rm = TRUE)
```

```
## 25%
##  24
```

Now we get back the 25% quantile value for weights. However, we often want to know more than one quantile. Luckily, the `probs` argument is **vectorised**, meaning it can take a whole vector of values. Let's try getting the 25%, 50% (median), and 75% quantiles all at once.

```
# calculate all quantiles

quantile(complete_old$weight, probs = c(0.25, 0.5, 0.75), na.rm = TRUE)
```

```
## 25% 50% 75%
##  24  42  53
```

While the `c()` function is very flexible, it doesn't necessarily scale well. If you want to generate a long vector from scratch, you probably don't want to type everything out manually. There are a few functions that can help generate vectors.

First, putting `:` between two numbers will generate a vector of integers starting with the first number and ending with the last. The `seq()` function allows you to generate similar sequences, but changing by any amount.

```
# generates a sequence of integers
1:10
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10
```

```
# with seq() you can generate sequences with a combination of:
#   from: starting value
#   to: ending value
#   by: how much should each entry increase
#   length.out: how long should the resulting vector be
seq(from = 0, to = 1, by = 0.1)
```

```
##  [1] 0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1.0
```

```
seq(from = 0, to = 1, length.out = 50)
```

```
##  [1] 0.00000000 0.02040816 0.04081633 0.06122449 0.08163265 0.10204082
##  [7] 0.12244898 0.14285714 0.16326531 0.18367347 0.20408163 0.22448980
## [13] 0.24489796 0.26530612 0.28571429 0.30612245 0.32653061 0.34693878
## [19] 0.36734694 0.38775510 0.40816327 0.42857143 0.44897959 0.46938776
## [25] 0.48979592 0.51020408 0.53061224 0.55102041 0.57142857 0.59183673
## [31] 0.61224490 0.63265306 0.65306122 0.67346939 0.69387755 0.71428571
## [37] 0.73469388 0.75510204 0.77551020 0.79591837 0.81632653 0.83673469
## [43] 0.85714286 0.87755102 0.89795918 0.91836735 0.93877551 0.95918367
## [49] 0.97959184 1.00000000
```

```
seq(from = 0, by = 0.01, length.out = 20)
```

```
##  [1] 0.00 0.01 0.02 0.03 0.04 0.05 0.06 0.07 0.08 0.09 0.10 0.11 0.12 0.13 0.14
## [16] 0.15 0.16 0.17 0.18 0.19
```

Finally, the `rep()` function allows you to repeat a value, or even a whole vector, as many times as you want, and works with any type of vector.

```
# repeats "a" 12 times
rep("a", times = 12)
```

```
##  [1] "a" "a" "a" "a" "a" "a" "a" "a" "a" "a" "a" "a"
```

```
# repeats this whole sequence 4 times
rep(c("a", "b", "c"), times = 4)
```

```
##  [1] "a" "b" "c" "a" "b" "c" "a" "b" "c" "a" "b" "c"
```

```
# repeats each value 4 times
rep(1:10, each = 4)
```

```
##  [1]  1  1  1  1  2  2  2  2  3  3  3  3  4  4  4  4  5  5  5  5  6  6  6  6  7
## [26]  7  7  7  8  8  8  8  9  9  9  9 10 10 10 10
```

Challenge 2.a: Creating sequences

1. Write some code to generate the following vector:

[-3 -2 -1 0 1 2 3 -3 -2 -1 0 1 2 3 -3 -2 -1 0 1 2 3]

```
rep(-3:3, 3)
```

```
##  [1] -3 -2 -1  0  1  2  3 -3 -2 -1  0  1  2  3 -3 -2 -1  0  1  2  3
```

```
# this also works
rep(seq(from = -3, to = 3, by = 1), 3)
```

```
##  [1] -3 -2 -1  0  1  2  3 -3 -2 -1  0  1  2  3 -3 -2 -1  0  1  2  3
```

```
# you might also store the sequence as an intermediate vector

my_seq <- seq(from = -3, to = 3, by = 1)
rep(my_seq, 3)
```

```
##  [1] -3 -2 -1  0  1  2  3 -3 -2 -1  0  1  2  3 -3 -2 -1  0  1  2  3
```

Challenge 2.b: Creating sequences

Calculate the quantiles for the `complete_old` hindfoot lengths at every 5% level (0%, 5%, 10%, 15%, etc.)

```
quantile(complete_old$hindfoot_length,
         probs = seq(from = 0, to = 1, by = 0.05),
         na.rm = T)
```

```
##   0%   5%  10%  15%  20%  25%  30%  35%  40%  45%  50%  55%  60%  65%  70%  75%
##    6   16   17   19   20   21   22   31   33   34   35   35   36   36   36   37
##  80%  85%  90%  95% 100%
##   37   39   49   51   70
```

2.5 Building with vectors

We have now seen vectors in a few different forms: as columns in a data.frame and as single vectors. However, they can be manipulated into lots of other shapes and forms. Some other common forms are:

- **matrices**
  - 2-dimensional numeric representations
- **arrays**
  - many-dimensional numeric
- **lists**
  - lists are very flexible ways to store vectors
  - a list can contain vectors of many different types and lengths
  - an entry in a list can be another list, so lists can get deeply nested
  - a data.frame is a type of list where each column is an individual vector and each vector has to be the same length, since a data.frame has an entry in every column for each row
- **factors**
  - a way to represent categorical data
  - factors can be ordered or unordered
  - they often *look* like character vectors, but behave differently
  - under the hood, they are integers with character labels, called **levels**, for each integer

2.5.1 Factors

We will spend a bit more time talking about factors, since they are often a challenging type of data to work with. We can create a factor from scratch by putting a character vector made using `c()` into the `factor()` function:

```
# create a factor

sex <- factor(c("male", "female", "female", "male", "female", NA))

sex
```

```
## [1] male   female female male   female <NA>
## Levels: female male
```

We can inspect the levels of the factor using the `levels()` function:

```
# inspect levels

levels(sex)
```

```
## [1] "female" "male"
```

The **`forcats`** package from the `tidyverse` has a lot of convenient functions for working with factors. We will show you a few common operations, but the `forcats` package has many more useful functions.

To find out more about the `forcats` package, use `?`.

```
?forcats
```

```
# change the order of the levels
fct_relevel(sex, c("male", "female"))
```

```
## [1] male   female female male   female <NA>
## Levels: male female
```

```
# change the names of the levels
fct_recode(sex, "M" = "male", "F" = "female")
```

```
## [1] M    F    F    M    F    <NA>
## Levels: F M
```

```
# turn NAs into an actual factor level (useful for including NAs in plots)
fct_na_value_to_level(sex, "(Missing)")
```

```
## [1] male      female    female    male      female    (Missing)
## Levels: female male (Missing)
```

In general, it is a good practice to leave your categorical data as a **character** vector until you need to use a factor. Here are some reasons you might need a factor:

1. Another function requires you to use a factor
2. You are plotting categorical data and want to control the ordering of categories in the plot

Since factors can behave differently from character vectors, it is always a good idea to check what type of data you're working with. You might use a new function for the first time and be confused by the results, only to realize later that it produced a factor as an output, when you thought it was a character vector.

It is fairly straightforward to convert a factor to a character vector:

```
# convert factor to character

as.character(sex)
```

```
## [1] "male"   "female" "female" "male"   "female" NA
```

However, you need to be careful if you're somehow working with a factor that has numbers as its levels:

```
# create a factor with numeric levels

f_num <- factor(c(1990, 1983, 1977, 1998, 1990))

# this will pull out the underlying integers, not the levels
as.numeric(f_num)
```

```
## [1] 3 2 1 4 3
```

```
# if we first convert to characters, we can then convert to numbers
as.numeric(as.character(f_num))
```

```
## [1] 1990 1983 1977 1998 1990
```

2.6 Assignment, objects, and values

We've already created quite a few objects in R using the `<-` assignment arrow, but there are a few finer details worth talking about. First, let's start with a quick challenge.

Challenge 3: Assignments and objects

What is the value of `y` after running the following code?

```
x <- 5
y <- x
x <- 10
```

```
x <- 5
y <- x
x <- 10
y
```

```
## [1] 5
```

Understanding what's going on here will help you avoid a lot of confusion when working in R. When we assign something to an object, the first thing that happens is the right hand side gets *evaluated*. The same thing happens when you run something in the console: if you type `x` into the console and hit `Enter`, R returns the value of `x`. So when we first ran the line `y <- x`, `x` first gets evaluated to the value of `5`, and this gets assigned to `y`. The objects `x` and `y` are not actually linked to each other in any way, so when we change the value of `x` to `10`, `y` is unaffected.

This also means you can run multiple nested operations, store intermediate values as separate objects, or overwrite values:

```
# first, x gets evaluated to 5
x <- 5

# then 5/2 gets evaluated to 2.5
# then sqrt(2.5) is evaluated
sqrt(x/2)
```

```
## [1] 1.581139
```

```
# we can also store the evaluated value of x/2
# in an object y before passing it to sqrt()
y <- x/2

sqrt(y)
```

```
## [1] 1.581139
```

```
# first, the x on the righthand side gets evaluated to 5
# then 5 gets squared
# then the resulting value is assigned to the object x

x <- x^2

x
```

```
## [1] 25
```

2.7 Naming objects

You will be naming a lot of objects in R, and there are a few common naming rules and conventions:

- make names clear without being too long
  - `wkg` is probably too short
  - `weight_in_kilograms` is probably too long
  - `weight_kg` is good
- names cannot start with a number
- names are case sensitive
- you cannot use the names of fundamental functions in R, like `if`, `else`, or `for`
  - in general, avoid using names of common functions like `c`, `mean`, etc.
- avoid dots `.` in names, as they have a special meaning in R, and may be confusing to others
- two common formats are `snake_case` and `camelCase`
- be consistent, at least within a script, ideally within a whole project
- you can use an R style guide like Google's (https://google.github.io/styleguide/Rguide.xml) or tidyverse's (https://style.tidyverse.org/)

Key points

- functions like `head()`, `str()`, and `summary()` are useful for exploring data.frames
- most things in R are vectors, vectors stitched together, or functions
- make sure to use `class()` to check vector types, especially when using new functions
- factors can be useful, but behave differently from character vectors